

MÓDULO 8: SEGURIDAD

INSECURE DESIGN

OWASP TOP 10

BIG school

GENERAMOS
NEGOCIO

BIGSEO

Insecure Design

Índice

- ¿Qué vamos a ver?
- ¿Qué es Insecure Design?
- ¿En qué consiste conceptualmente?
- Ejemplos típicos
- ¿Qué puede hacer alguien que aproveche un diseño inseguro?
- ¿Cómo detectarlo?
- ¿Cómo prevenir Insecure Design?
- ¿Cómo mitigarlo cuando ya está en producción?
- ¿Cómo integrar buenas prácticas de diseño en equipos pequeños?
- Checklist práctico para diseñadores/devs
- Conclusiones



Insecure Design

BIG school

BIGSEIO

GENERAMOS
NEGOCIO

¿Qué vamos a ver?

- Vulnerabilidad que proviene del diseño, no del código.
- Surge de decisiones arquitectónicas o de producto inseguras.
- Veremos qué es, ejemplos, impacto, detección y prevención desde el diseño.

¿Qué es Insecure Design?

- Fallos en arquitectura, requisitos o flujos de negocio que no contemplan seguridad.
- El sistema es vulnerable, aunque el código esté bien implementado.
- Ejemplos: funcionalidades inseguras, roles mal definidos, permisos sin control.
- No es un bug: es una omisión o mala decisión de diseño.

¿En qué consiste conceptualmente?

- Requisitos sin criterios de seguridad.
- Arquitectura que amplifica riesgos (servicios con excesos de permisos).
- Faltan principios de "least privilege" y "defense in depth".
- Ausencia de trazabilidad o separación de funciones.

Resultado: cada componente puede funcionar bien, pero el sistema es inseguro.

Ejemplos típicos

- Exposición de datos por diseño: enlaces compartidos sin expiración.
- Roles mal definidos: "usuario" y "admin" sin granularidad.
- Excesiva delegación a terceros: sin límites ni scopes claros.
- Sin rate limiting: APIs que permiten scraping o fuerza bruta.
- Dependencias sin plan de actualización: riesgo permanente.
- UX sobre seguridad: autenticación opcional en funciones críticas.
- Falta de auditoría: sin logs de acciones sensibles.

¿Qué puede hacer alguien que aproveche un diseño inseguro?

Acciones sensibles por flujos legítimos.

- Escalada de privilegios por reglas de negocio mal definidas.
- Abuso de integraciones de terceros.
- Scraping o abuso automatizado sin limitaciones.
- Dificultad para detectar incidentes sin trazabilidad.

Impacto amplio: compromete seguridad y dificulta respuesta a incidentes.

¿Cómo detectarlo?

Durante requisitos:

- Revisar decisiones que ignoran seguridad. Threat modeling; su ausencia o superficialidad revela riesgo. En revisiones de arquitectura; verificar principios de seguridad.

En desarrollo: detectar mitigaciones improvisadas como indicios de diseño débil.

En testing: validar lógica de negocio y flujos combinados indebidos.

En producción: monitorear abuso, accesos y permisos anómalos.

¿Cómo prevenir Insecure Design?

Principios de diseño:

- Aplicar threat modeling temprano y continuo.
- Incorporar garantías de seguridad en requisitos.
- Diseñar con mínimo privilegio y defensa en profundidad.
- Denegar por defecto, permitir solo lo necesario.
- Registrar trazabilidad y auditar desde el diseño.

Prácticas concretas:

- Añadir criterios de seguridad en historias de usuario.
- Definir scopes, límites y contratos seguros para APIs.
- Plan de rotación y actualización de dependencias.
- Diseñar expiración y revocación de recursos.
- Automatizar tests de lógica de negocio.
- Establecer métricas de seguridad y SLAs.
- Revisiones formales de arquitectura con seguridad.

¿Cómo mitigarlo cuando ya está en producción?

- Evaluar riesgos y alcance del diseño inseguro.
- Aplicar controles temporales (WAF, rate limiting, ACL).
- Parches rápidos en validación o middleware de autorización.
- Revocar permisos excesivos y rotar secretos comprometidos.
- Invalidar recursos inseguros (enlaces, tokens).
- Planificar rediseño progresivo (separación de servicios, scopes).
- Monitorear comportamiento tras los cambios y documentar aprendizajes.
- La clave es contener impacto mientras se trabaja el rediseño.

¿Cómo integrar buenas prácticas de diseño en equipos pequeños?

- Proponer mini threat modelings de 15–30 minutos por feature.
- Crear checklists simples de seguridad en la definición de done.
- Escribir pruebas que validen reglas de negocio críticas.
- Ser security champion compartiendo patrones seguros en el equipo.
- Documentar decisiones sobre permisos y scopes.
- Aplicar expiraciones, scopes limitados y logging básico.

Pequeñas acciones mejoran la madurez de seguridad del proyecto.

Checklist práctico para diseñadores/devs

1. ¿Se hizo al menos un mini threat modeling para la feature?
2. ¿Roles, scopes y permisos definidos con suficiente granularidad?
3. ¿Existen límites de acceso (rate limit, paginación, throttling)?
4. ¿Los recursos compartidos tienen expiración o revocación?
5. ¿Integraciones externas con scopes mínimos y auditoría?
6. ¿Se incluyen logs y trazabilidad en funciones críticas?
7. ¿Plan de rotación para claves y dependencias?
8. ¿Tests que validen reglas de seguridad del negocio?
9. ¿Plan de mitigación si falla una suposición de diseño?

Conclusiones

1. Insecure Design no se corrige con parches, se previene con pensamiento crítico.
2. El rol del desarrollador junior es cuestionar, proponer y validar seguridad desde el diseño.
3. Pequeñas decisiones —como expiraciones automáticas o test de permisos— evitan grandes incidentes.

**Diseñar con seguridad es pensar a largo plazo:
crear software que resista el cambio y el abuso.**